AI ASSISTED CODING

NAME: J.Shyamsundar

ROLL.NO:2303A52384

BATCH-13

LAB-2

PROMPT: generate a python code for happy numbers.

TOOLS USED:

S.No.	Tool	Purpose
1	ChatGPT	AI-assisted coding, explanation and debugging
2	Claude	AI-assisted code generation and reasoning
3	Google Gemini	AI-assisted coding and code explanation
4	Google Colab	Cloud-based Python coding and execution

1.CHAT GPT:

```
def is_happy(n):
    seen = set()
```

```
    while n != 1 and n not in seen:
        seen.add(n)
        n = sum(int(digit) ** 2 for digit in str(n))

    return n == 1



num = int(input("Enter a number: "))

if is_happy(num):
    print(num, "is a Happy Number")
else:
    print(num, "is not a Happy Number")
```

2.CLAUDE:

```
num = int(input("Enter a number: "))

while num != 1 and num != 4:
    sum = 0

    while num > 0:
        digit = num % 10
```

```
        sum += digit * digit
        num //= 10

    num = sum

if num == 1:
    print("Happy Number")
else:
    print("Not a Happy Number")
```

3.GOOGLE GEMINI:

```
def is_happy(n: int) -> bool:
    """Checks if a given number is a Happy Number."""
    seen = set()
    while n != 1 and n not in seen:
        seen.add(n)
        # Calculate sum of squares of digits
        n = sum(int(digit) ** 2 for digit in str(n))
    return n == 1


def generate_happy_numbers(limit: int) -> list[int]:
```

```
    """Generates all Happy Numbers up to a specified limit."""
    return [num for num in range(1, limit + 1) if is_happy(num)]


# --- Usage Examples ---

# 1. Test individual numbers
test_numbers = [19, 7, 4]
for num in test_numbers:
    status = "Happy" if is_happy(num) else "Unhappy"
    print(f"{num} is {status}")

# 2. Find all happy numbers up to 100
happy_up_to_100 = generate_happy_numbers(100)
print("\nHappy Numbers up to 100:")
print(happy_up_to_100)
```

4.GOOGLE COLLAB:

```
def is_happy(n: int) -> bool:
    """Checks if a given number is a Happy Number."""
    seen = set()
```

```
    while n != 1 and n not in seen:
        seen.add(n)
        # Calculate sum of squares of digits
        n = sum(int(digit) ** 2 for digit in str(n))
    return n == 1


def generate_happy_numbers(limit: int) -> list[int]:
    """Generates all Happy Numbers up to a specified limit."""
    return [num for num in range(1, limit + 1) if is_happy(num)]


# --- Usage Examples ---

# 1. Test individual numbers
test_numbers = [19, 7, 4]
for num in test_numbers:
    status = "Happy" if is_happy(num) else "Unhappy"
    print(f"{num} is {status}")
```

```
# 2. Find all happy numbers up to 100
happy_up_to_100 = generate_happy_numbers(100)
print("\nHappy Numbers up to 100:")
print(happy_up_to_100)
```

Comparative Analysis

Criteria	ChatGPT	Claude	Google Gemini	Google Colab
Code Generation	Excellent	Excellent	Excellent	Good
Code Correctness	Excellent	Excellent	Excellent	—
Code Readability	Excellent	Excellent	Excellent	—
Code Explanation	Excellent	Excellent	Excellent	—
Test Cases	Provided	Provided	Provided	Executes tests
Code Execution	Not the main purpose	Not the main purpose	Not the main purpose	**Excellent**
Debugging	Excellent	Excellent	Excellent	Good
Documentatio	Good	Excellen	Excellent	—

Criteria	ChatGPT	Claude	Google Gemini	Google Colab
n		t		
Type Hints	Not included	Not included	Included	—
Complexity Analysis	Limited	Limited	Included	—
Ease of Use	Excellent	Excellent	Excellent	Excellent
Main Role	AI coding assistant	AI coding assistant	AI coding assistant	Coding & execution environment